Inventory Monitoring System Architecture

1. Solution Overview

The proposed solution is a **modular IoT-based inventory monitoring platform** designed for deployment across **small bins, medium containers, drums, and pallet-scale inventory points** within large facilities.

The system is designed to support:

- **Hundreds of weighing nodes per facility**
- **Low-power battery operation**
- **Scheduled measurements 2–3 times per day**
- **Accurate and stable calibration management**
- **Inventory alerts and reporting**
- **ERP integration through configurable file export / FTP workflow**

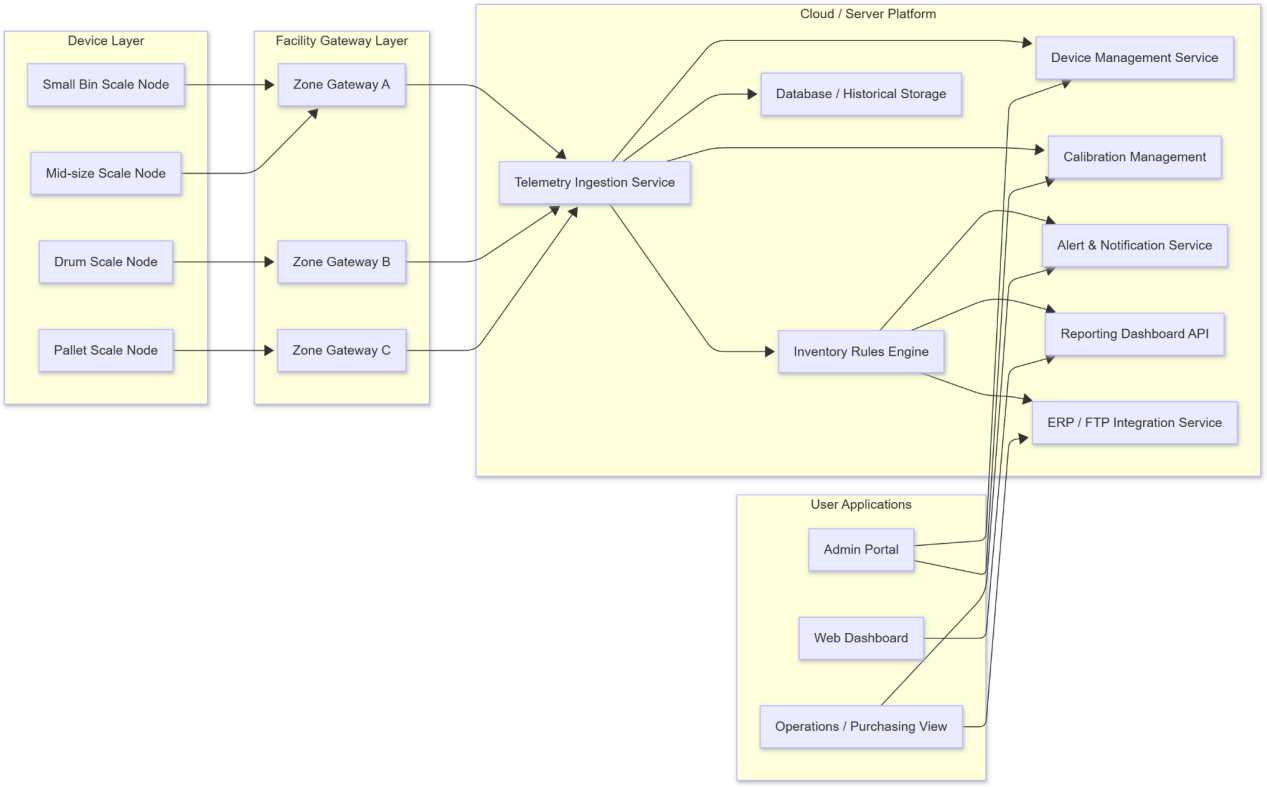
The recommended architecture is based on three layers:

1. **Smart Scale Nodes**
2. **Facility Gateways**
3. **Cloud / Server Software Platform**

This approach provides the best balance between **scalability, battery life, deployment flexibility, and cost optimization**

.

2. High-Level System Architecture



3. Architecture Details

A. Smart Scale Nodes

Each scale location is equipped with a **battery-powered weighing node** that measures weight at scheduled intervals and transmits the result wirelessly.

This allows:

- simple installation
- minimal wiring
- low maintenance
- easy scaling across a facility

B. Gateway Layer

Instead of having every scale connect directly to the internet, scale nodes communicate with **local facility gateways**.

This provides:

- better scalability for hundreds of devices
- lower power consumption at the node
- easier network control inside large warehouses

- simpler deployment in environments where Wi-Fi coverage may not be consistent at every scale position

C. Cloud / Server Platform

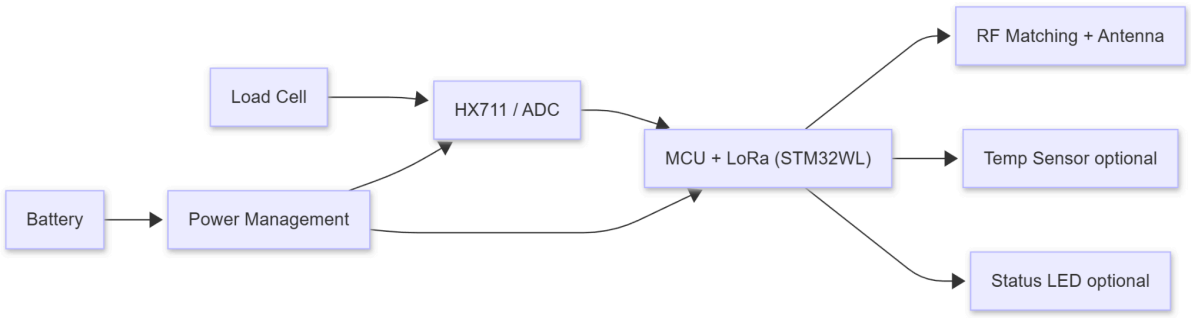
The backend platform handles:

- weight data collection
- calibration profiles
- inventory conversion logic
- low-stock alerts
- historical reporting
- ERP export / FTP file generation

This keeps the node firmware simple and enables business logic to evolve without redesigning hardware.

4. Smart Scale Node Architecture

The device design should follow a **shared electronics platform** strategy so the same core PCB can be reused across multiple scale sizes.



Core node functions

- wake on schedule
- capture multiple samples
- filter and stabilize measurement
- apply calibration parameters
- transmit data packet to gateway
- return to low-power sleep mode

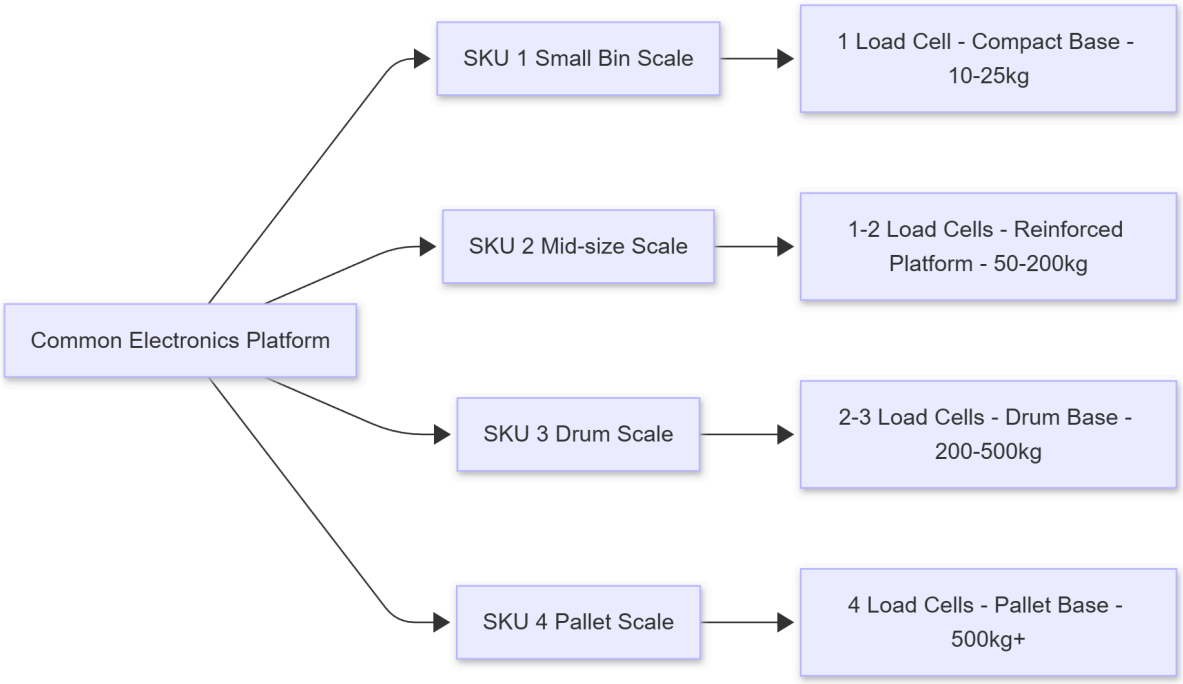
Shared hardware strategy

The same electronics core can be reused while only changing:

- load cell capacity
- mechanical platform
- enclosure / mounting arrangement

5. Product Family Strategy

To meet both cost and coverage requirements, the system should be structured as a **single electronics platform with multiple mechanical SKUs**.



Single load cell is recommended for small bin applications due to cost and simplicity.

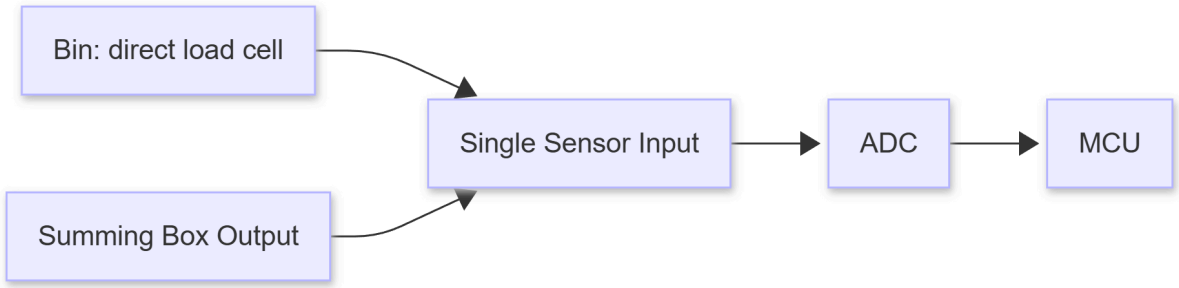
Multi-load-cell configurations (2–4 sensors) are required for larger platforms such as drums and pallets to ensure accuracy under uneven load distribution.

Type	Loadcell	Note	Summing box
Bin	1	Low cost, simple	No
Mid	1-2	Moderate stability	Yes
Drum	2-3	Uneven load	Yes
Pallet	4	Industrial standard	Yes

The system uses a **common PCB with a single load-cell interface** across all product types.

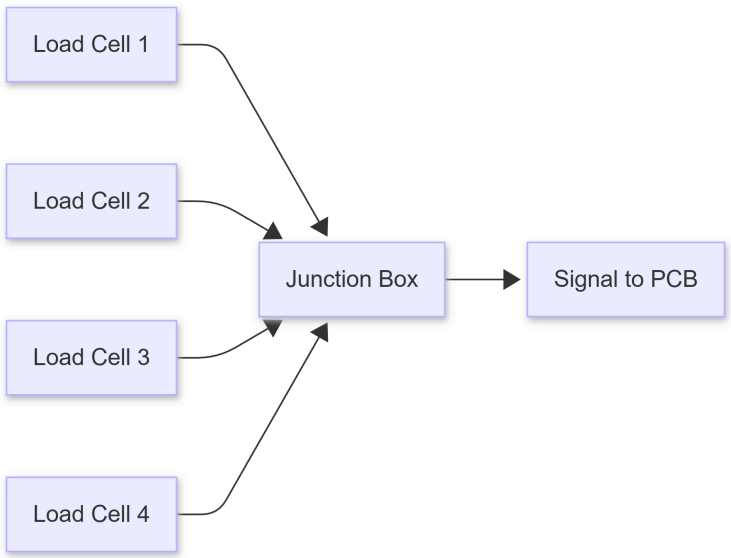
- **Single load cell (bin, small scale):** connected directly to the PCB

- **Multiple load cells (drum, pallet):** combined through a **summing (junction) stage** before connecting to the same PCB input



Summing/Junction Function

The summing junction combines signals from multiple load cells into a single differential signal.



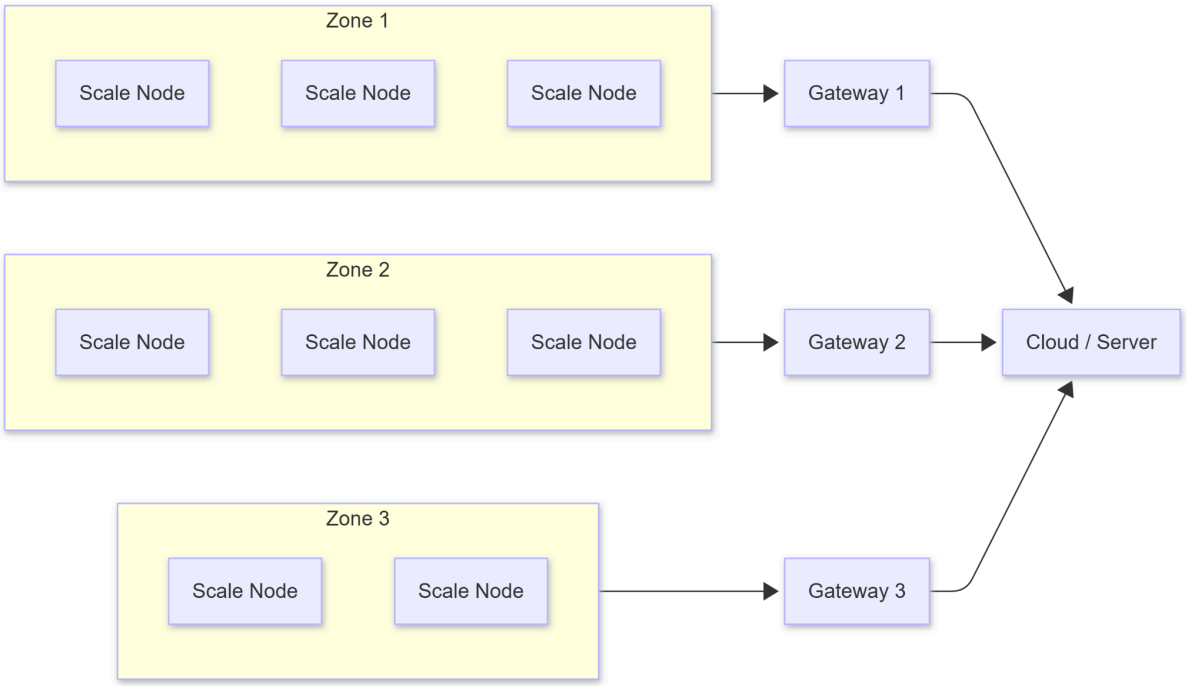
Accuracy

- The summing stage introduces **negligible error (~0.01–0.05% FS)**
- Overall system accuracy is mainly affected by:
    - mechanical structure
    - load distribution
    - calibration quality

The initial implementation will focus on the **small bin scale configuration (single load cell)** only.

6. Facility Communication Architecture

For large facilities, the most practical deployment model is to group devices by zone and assign each zone one gateway.

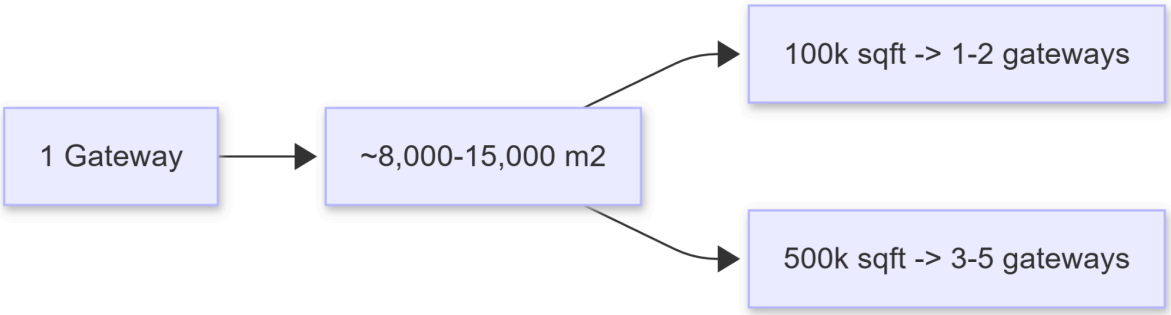


Recommended communication model

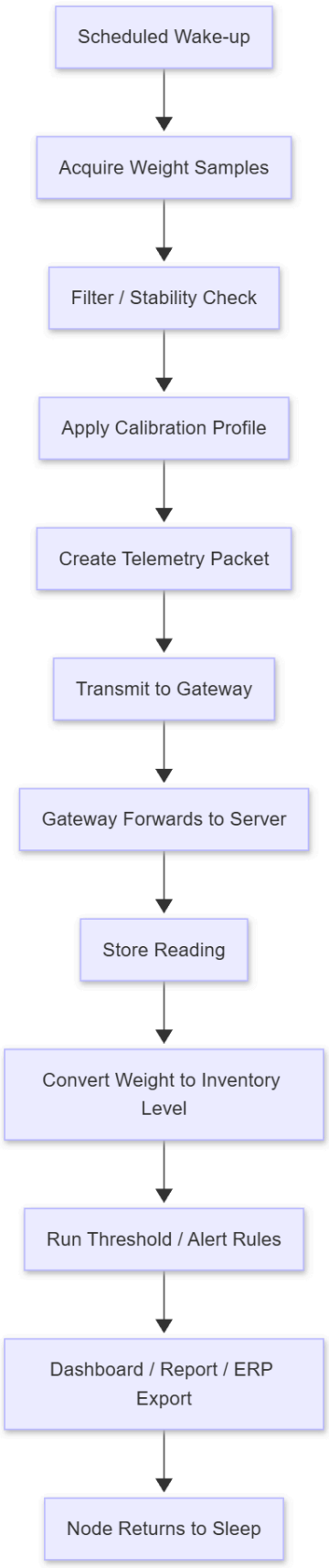
For MVP and cost-focused rollout, the recommended structure is:

- **Scale Node → Gateway:** short-range low-power wireless
- **Gateway → Cloud:** Ethernet / Wi-Fi / Cellular

The final wireless protocol can be selected based on facility layout and RF survey, but the architecture should stay the same.



7. Data Flow Architecture

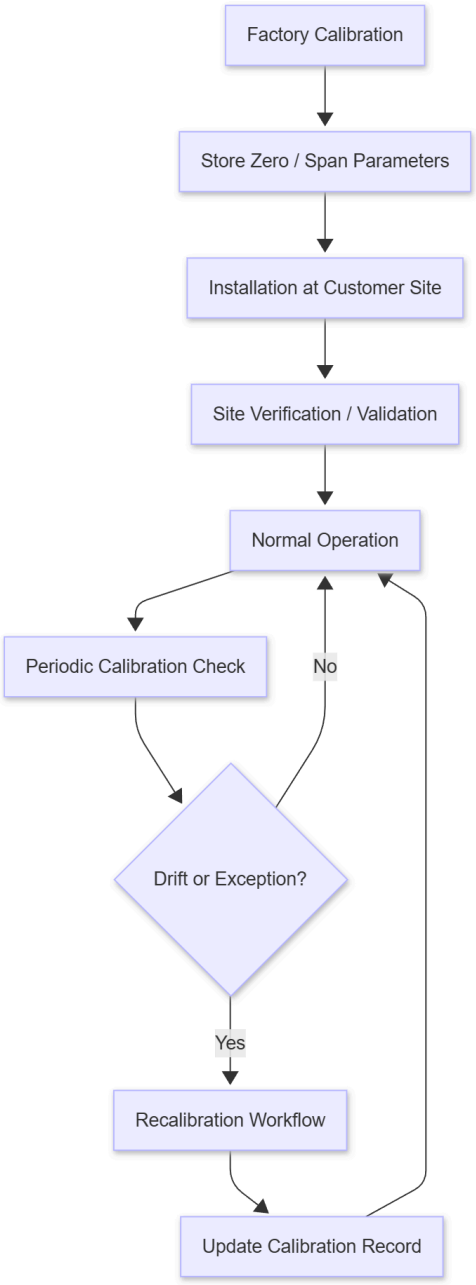


Telemetry content

A typical reading should include:

- device_id
- timestamp
- raw_weight
- calibrated_weight
- battery_level
- temperature optional
- firmware_version
- calibration_version
- RSSI/link quality optional

8. Calibration Architecture



Calibration is one of the most critical success factors in this solution.
It should be treated as a full subsystem, not just a one-time device setting.

Calibration mode

Factory calibration

- zero offset
- span calibration
- hardware profile assignment

Site calibration / verification

- confirm container setup
- verify tare weight
- verify actual installation conditions

Periodic maintenance

- software-driven reminders
- drift detection rules
- recalibration log and audit trail

The customer's main concern is not just reading weight, but ensuring the system remains **accurate over time**.
The architecture should therefore include:

- calibration version control
- calibration history
- service workflow support
- exception detection

Calibration outputs

- zero offset
- span / gain
- tare weight
- calibration version
- calibrated by
- calibrated time

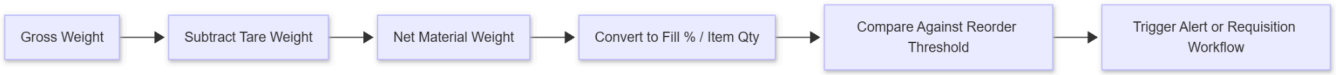
Storage

- local device storage
- backend database

Recalibration triggers

- drift exceeds threshold
- container change
- load cell replacement
- scheduled maintenance

9. Backend Inventory Logic Architecture



The backend transforms raw weight measurements into actionable inventory data, enabling automated monitoring and replenishment workflows.

Configuration Inputs

The inventory logic relies on configurable parameters per device/item:

- tare weight
- unit weight
- full capacity
- reorder threshold

Calculation Logic

net_weight = gross_weight - tare_weight
quantity = net_weight / unit_weight
fill_percentage = net_weight / full_capacity

Typical conversion logic

- gross weight
- minus empty container tare
- equals net material weight
- converted to:
    - quantity
    - percentage remaining
    - reorder status

This is especially useful for:

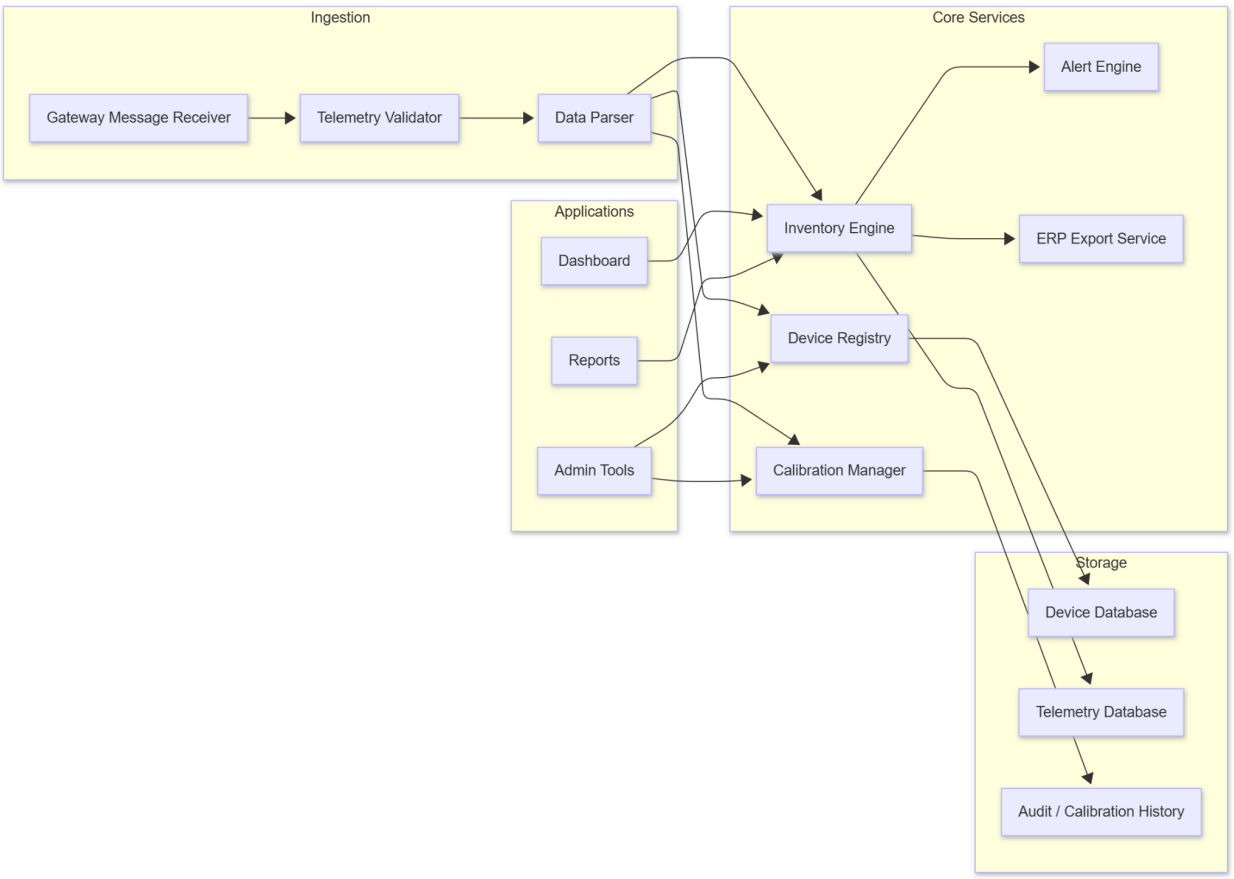
- bins with small parts
- drums with consumables
- palletized materials

10. Software Backend Platform Architecture

Overview

The backend platform is structured into **three main layers**:

- Ingestion Layer
- Core Services
- Storage & Applications



Main backend modules

- Device Management
- Calibration Management
- Telemetry Storage
- Inventory Rules Engine
- Alerting
- Reporting
- ERP/FTP Integration

Ingestion Layer

Responsible for receiving and preparing device data.

Components:

- Gateway Message Receiver
- Telemetry Validator
- Data Parser

Functions:

- Receive telemetry from gateway
- Validate payload (device_id, timestamp, format)
- Parse data into internal structure

Core Services

The main business logic layer.

Inventory Engine

- Convert weight → quantity / fill level
- Apply item configuration
- Provide inventory state

Device Registry

- Manage device metadata
- Map device → location / item

Calibration Manager

- Store and manage calibration parameters
- Apply calibration to incoming data

Alert Engine

- Detect low stock / abnormal conditions
- Trigger alerts

ERP Export Service

- Generate ERP-compatible files
- Handle FTP/SFTP upload

Storage Layer

Stores all system data.

Components:

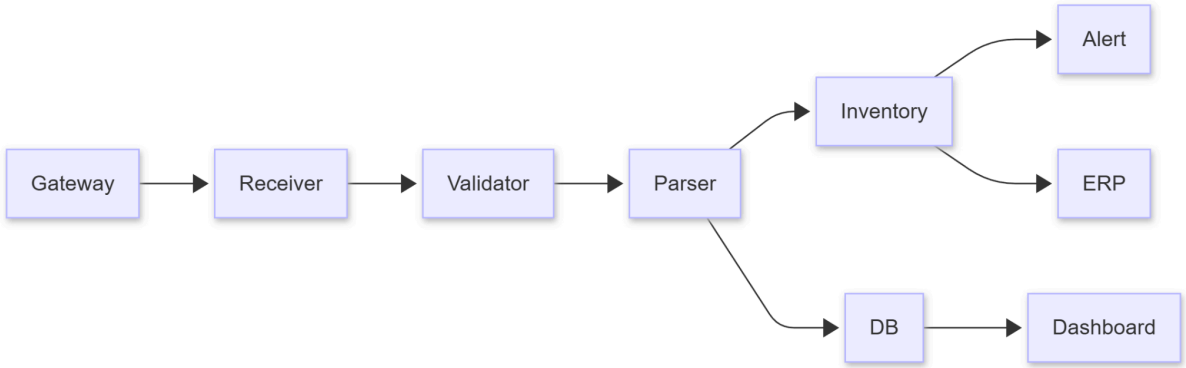
- Device Database
- Telemetry Database
- Audit / Calibration History

Application Layer

User-facing components.

- Dashboard → real-time monitoring
- Reports → historical data
- Admin Tools → configuration (mapping, calibration, thresholds)

Data Flow



11. ERP Integration Approach

Integration Model

The system integrates with ERP via **file-based export (FTP/SFTP)** in the MVP phase.

Data Flow:

1. Scale node sends weight data
2. Backend converts to **inventory values**
3. Backend generates **export file**
4. File is uploaded to ERP via FTP/SFTP
5. ERP system processes file and updates inventory / triggers actions

Responsibilities:

System Responsibilities

- Convert weight → inventory (quantity / %)
- Map device data to ERP fields
- Generate export files
- Upload files to ERP (FTP/SFTP)
- Handle retry and logging

Client Responsibilities

- Provide ERP format specification
- Provide integration method (FTP/API)
- Configure ERP to receive and process data

Field Mapping (Configurable)

Field	Source
device_id	sensor
part_number	configuration
quantity	calculated
unit	configuration
location	zone
status	system logic

Output Generation

Backend generates configurable output files based on:

- device group
- customer site
- item type
- reorder threshold
- ERP field mapping

Export file outputs structure: This ERP data format will be defined as below or Provided by the client.

- CSV
- TXT

Example:

timestamp,device_id,part_number,quantity,unit,location,status

2026-04-10T10:00:00Z,BIN_001,BEARING_A,120,pcs,ZONE_A,OK

2026-04-10T10:00:00Z,DRUM_002,CHEM_X,45,kg,ZONE_B,LOW

Data Delivery

- Scheduled file upload via **FTP/SFTP: every X minutes / hours or typical: 1–4 time/day**
- Configurable export intervals (e.g., periodic or event-based): **when low stock or big event (define later)**

FTP Configuration

Configurable parameters: **Example filename: inventory_20260410_1000.csv**

- server IP / domain
- port
- username / password
- protocol: FTP / SFTP
- folder path
- file naming format

Error Handling

- retry if upload fail
- log file status

- Avoid duplicate (timestamp + ID)
- store queue file if offline

12. Power Strategy

2–3 times per day and accepts **1-year battery replacement**, the power architecture can be designed conservatively and cost-effectively.

Device behavior

- sleep most of the time
- wake on RTC schedule
- enable sensing only during measurement
- transmit in short bursts
- return to sleep
- AA battery

13. Failures cases

Sensor Node

- Read wrong value → retry
- Low battery → flag
- Not connect gateway→ retry

Gateway

- Lost internet → buffer, local store
- duplicate packet → filter

Backend

- packet error→ reject
- not recognize device→ ignore/log

14. MVP Architecture

MVP scope

- common electronics platform
- small bin scale node
- one gateway design
- backend for telemetry, dashboard, alerts
- calibration workflow
- basic ERP/FTP export

15. Executive Summary

The architecture is a **modular, gateway-based, low-power IoT weighing platform** designed to support a wide range of inventory monitoring use cases in large facilities.

Key strengths of this architecture

- scalable to hundreds of devices per facility
- battery-friendly for 2–3 measurements per day
- supports annual battery replacement model
- optimized for cost through shared electronics
- allows structured calibration management
- supports alerts, reporting, and ERP integration
- provides a practical path from MVP to production rollout